

DDoS Resilience Testing

Revision: 2 Last modified: 2026-08-18T19:25:00Z

Documents challenges/scripts/ddos_resilience_challenge.sh” the DDoS-class testing scaffold added by **BOB-074** to close a gap in boba’s §11.4.27 mandated test-type matrix (see docs/testing/test_type_matrix.md for the full audit). §11.4.85 (stress + chaos mandate) and §11.4.27 (100% test-type coverage, DDoS is one of the enumerated types) are the governing anchors.

What it tests

boba exposes three public, unauthenticated-by-default HTTP surfaces:

Endpoint	Port	Probed path	Why it matters
Merge search (FastAPI/uvicorn)	7187	GET /health	Fans out to up to 43 trackers on POST /api/v1/search” the expensive, real DDoS target (see “Scoping decisions” below for why the fanout path itself is never driven)
qBittorrent WebUI	7185	GET /	network_mode: host, directly reachable from the host network
Boba Jackett API (Go/Gin)	7189	GET /healthz	Owns Jackett credentials + indexer overrides, backed by encrypted SQLite

For each reachable endpoint the challenge drives a bounded, localhost-only curl-parallel-loop (primary” tool-independent) plus ab/ApacheBench (supplementary evidence, when installed) across three concurrency tiers, and asserts:

1. **Crash resistance**” zero 5xx responses and zero connection failures (including client-side timeouts under load” see “Terminology” below) across all three tiers.
2. **Rate limiting**” a 429 (or equivalent) appears once load is heavy enough. See “RED/GREEN polarity” below” this is currently

an honest SKIP because **no rate limiter exists anywhere in boba™s stack.**

3. **Cross-endpoint isolation** “ while one endpoint is under its heaviest tier, the other two stay responsive (2xx within a bounded timeout).

Terminology: “crash” includes client-side timeout

The task brief™s crash-resistance criterion is “return 5xx or connection-reset.” This challenge additionally counts a client-side --max-time timeout (curl exit code 28) as a crash-resistance failure. From the caller™s perspective, a backend that never answers within a reasonable window under load is exactly as unavailable as one that resets the connection “ and (see “Findings” below) this distinction is what caught a real defect. The per-tier output breaks timeouts out separately (timeout=N other=M) so the two failure modes are never conflated when reading results.

Bounded chaos knobs (§12.6 / §12.11 host-safety)

Knob	Value	Rationale
Concurrency tiers	10, 25, 50	Modest ramp; ab refuses -c > -n, so REQUESTS_PER_TIER must be % the heaviest tier
Requests per tier	50	Matches the heaviest concurrency tier (see above)
Per-request timeout	3s	curl --max-time; also fed to ab -s
Per-tier wall-clock budget	12s	Hard early-stop “ a genuinely slow backend gets a smaller sample , never a longer-running script (see run_curl_status_probe)
Cross-endpoint probe timeout	3s	
Scope	127.0.0.1 only, hardcoded	No host override exists “ this challenge cannot be pointed at anything but localhost

Measured wall-clock for a full live run (2026-08-18, 8-core/62G dev host, stack already running): ~**17s** when all three endpoints are warm, ~**55s** in the run that caught the boba-jackett cold-start finding below. Both are comfortably inside run_all_challenges.sh™s 180s per-script timeout.

The challenge never targets anything but 127.0.0.1, never issues a destructive command, and its `--self-validate` mode exercises two THROWAWAY local python3 HTTP servers “ never the real boba stack.

RED/GREEN polarity (§11.4.115)

The **rate-limiting** assertion is the one with real, meaningful polarity today (the crash-resistance and cross-endpoint checks are real pass/fail checks regardless of RED_MODE “ see “Findings” for what they actually caught).

- **RED_MODE=1** (reproduce the defect): asserts the **absence** of any 429 response under the heaviest tier. This currently **PASSes** for all three endpoints “ proving the challenge correctly reproduces boba’s real, current, verified state: **no rate-limit middleware, no slowapi-style Python limiter, no throttle in either Go service, and no nginx/reverse-proxy layer exists anywhere in the stack** (grepped 2026-08-18 across `download-proxy/src/`, `qBitTorrent-go/internal/`, and `docker-compose.yml`).
- **RED_MODE=0** (default, GREEN guard): the **same** absence-of-429 observation is honestly **SKIPPed** (reason `extension absent`) rather than bluffed as a **PASS** or unfairly marked **FAIL** “ §11.4.6 forbids inventing a threshold nothing in the codebase enforces.

Real polarity going forward: the day a rate limiter is configured for any of the three endpoints, RED_MODE=0 starts asserting 429s actually appear under load (a genuine **PASS/FAIL** on whether the configured limit works), and RED_MODE=1 starts **FAILing** (correctly signalling the defect this anchor targets has been fixed). If that rate limiter is later stripped again, RED_MODE=1 goes back to **PASS** (defect reproduced) and RED_MODE=0 would **FAIL** “ “strip rate limiting “ challenge **FAILs**” holds from the point a limiter first exists.

Self-validation (§11.4.107(10)-style)

`bash challenges/scripts/ddos_resilience_challenge.sh --self-validate` spins up two throwaway local python3 `ThreadingHTTPServers` “ one that always answers 200, one that always answers 500 “ and asserts the crash-resistance detector correctly **PASSes** the good fixture and **FAILs** the bad one. This proves the detector is not a tautology (it can actually see a broken backend), not merely that it runs without crashing.

Scope limitation, stated honestly: this round only self-validates the crash-resistance detector (assertion 1). The rate-limiting detector (assertion 2) has no synthetic golden-bad fixture in this scaffold “ a throwaway server that emits 429s past some threshold would be needed to validate it, and was left out of this round for time. Filed as a followup (see below).

Scoping decisions

The real fanout path, `POST /api/v1/search`, is never driven by this challenge. Hammering it at any of the tested concurrency tiers would flood up to 43 **real third-party tracker sites** – risking IP bans on shared trackers, violating good-netizen practice, and doing real off-host damage that no localhost-only chaos budget can bound (Â§12.6/Â§12.11 explicitly scope host-safety to *this* host; they say nothing about the blast radius on someone else’s server, and common sense fills that gap). `GET /health` stands in as – is the merge-search process up and answering – the crash-resistance signal Â§11.4.85 needs – without touching the expensive orchestration path.

Followup filed: a sandboxed/mock-tracker variant of this challenge that exercises the real search-orchestration code path (dedup, enrichment, SSE streaming) under load without ever making a real outbound tracker request – e.g. – by pointing trackers= at a disabled/nonexistent tracker name (confirmed during authoring: `POST /api/v1/search` returns 404 Not Found for a GET probe since it’s POST-only, so this needs a real POST body to explore properly) or by standing up a local mock tracker HTTP server the merge-search’s tracker registry can be pointed at for the duration of a test run.

Findings (2026-08-18, from authoring this challenge)

Authoring this scaffold immediately surfaced two real gaps – consistent with Â§11.4.238’s mandate that automated QA be the discovery channel:

1. No rate limiting anywhere (see – RED/GREEN polarity – above)

Verified by source inspection across the whole stack – no `slowapi/limiter/throttle` import in `download-proxy/src/`, no `rate-limit` middleware in `qBitTorrent-go/internal/middleware/` (only `cors.go` + `logging.go` exist there) or `internal/jackettapi/` (only `auth_middleware_test.go` + `cors_middleware_test.go` exist – no `rate` middleware), and no `nginx/reverse-proxy` service in `docker-compose.yml`.

Candidate remediation approaches (documented, not implemented – out of this task’s file scope):

- A lightweight `nginx`-in-container reverse proxy fronting `:7185/:7187/:7189` with `limit_req_zone` – the most portable single fix, but adds a new container to the compose stack.
- A FastAPI dependency (e.g. – `slowapi`) for the merge-search service specifically, since it already sits behind `uvicorn`.
- A Gin `rate-limit` middleware for `boba-jackett` (Go), following the existing `internal/middleware/` pattern (`cors.go`, `logging.go`).

- qBittorrent™'s own WebUI has a documented failed-login ban list (bandwidth-shaping "Rate limiting" settings exist in the WebUI but were **not verified** during this task to cover request-rate, only bandwidth "do not assume this closes the gap without checking).

2. boba-jackett™'s /healthz amplifies under a cold-start burst "FIXED + live-verified (BOB-112, task #74)

Status update (2026-08-18, task #74): the 30s TTL cache recommended in fix option 1 below was implemented as commit 91b52db and then **live-verified with real wrk load-test evidence** "the followup this finding originally filed. See docs/qa/BOB-112/summary.md for the full before/after numbers: a §1.1 mutation reproducing this exact pre-fix code measured **97.1% client-side timeouts** (400/412 requests) at 4 threads — 100 connections — 30s against :7189/healthz; the real committed fix measured **0.0% timeouts** across 812,149 requests in the same window (27,049 req/s, p99 19.89ms) and collapsed upstream Jackett.GetCatalog() calls from one-per-request down to 2 calls in the entire 30s run. This challenge script now also ships a standing --healthz mode (see "Running manually" below) that re-runs a bounded version of this same check on every future invocation, so a regression that re-introduces the uncached call path trips a real FAIL, not just a filed finding.

The pre-fix root cause is preserved below for historical/forensic context.

Root cause: qBitTorrent-go/internal/jackettapi/health.go:60-63 "

```
if d.Jackett != nil {
    _, err := d.Jackett.GetCatalog()
    out.JackettOk = err == nil
}
```

HandleHealth makes a **synchronous, uncached** call to Jackett™'s catalog endpoint on **every single hit** to /healthz. There is no cache, no timeout distinct from the Jackett client™'s own default, and no circuit breaker.

Observed evidence (three independent live runs, RED_MODE=0, 2026-08-18):

```
--- boba_jackett: Boba Jackett API :7189 ---
    reachable: HTTP 200
    tier c=10 n=50: 5xx=0 conn_fail=0 (timeout=0 other=0) 429=0 elapsed
    tier c=25 n=50: 5xx=0 conn_fail=48 (timeout=48 other=0) 429=0 elapsed
    tier c=50 n=50: 5xx=0 conn_fail=50 (timeout=50 other=0) 429=0 elapsed
[FAIL] boba_jackett: 0 5xx + 98 connection failures out of 150 requests
```

98/150 (65%) of health-check requests timed out at the 3-second budget during this run. **Two subsequent runs against the same live process did not reproduce this** "all three tiers completed in ~2s each with zero failures once the initial burst had passed:

```
tier c=10 n=50: 5xx=0 conn_fail=0 (timeout=0 other=0) 429=0 elapsed=2
tier c=25 n=50: 5xx=0 conn_fail=0 (timeout=0 other=0) 429=0 elapsed=1
tier c=50 n=50: 5xx=0 conn_fail=0 (timeout=0 other=0) 429=0 elapsed=2
[PASS] boba_jackett: 0/150 requests returned 5xx or failed to connect ac
```

This is honestly reported as an intermittent, cold-state-dependent finding, not a deterministic one – per §11.4.7 (demotion requires same-conditions evidence), the recovery does NOT retract the finding; it narrows it. The trigger condition (a concurrent burst against a /healthz route that has not been recently exercised) is real and reproducible enough to have appeared in this task’s very first live run, but this scaffold does not currently force a guaranteed-cold state (that would require restarting the boba-jackett container as part of a routine test run – a more invasive chaos action deliberately left out of this scaffold pending further review; see followups).

Impact if left unfixed: an attacker (or even a mis-tuned external health-monitor hitting /healthz too aggressively) could make the Jackett management API’s own health surface appear down without ever touching Jackett itself – a genuine self-inflicted amplification vector, exactly the class of defect §11.4.85’s DDoS/chaos discipline exists to catch.

Recommended fixes (documented, not implemented – qBittorrent-go/internal/jackettapi/health.go is out of this task’s file scope):

1. Cache the Jackett liveness signal with a short TTL (e.g. 10-30s), refreshed by a background ticker rather than per-request.
2. Add a tight timeout + circuit breaker around the GetCatalog call so /healthz itself never blocks past ~250-500ms regardless of Jackett’s real state.
3. Make jackett_ok best-effort / asynchronously refreshed rather than synchronously computed on the request path.

Because the running boba-jackett process fully recovered (confirmed via podman ps – both jackett and boba-jackett containers stayed Up (healthy) throughout, and a post-test manual probe returned HTTP 200 in 0.51s), this is a **resilience/availability defect, not a crash** – the process itself never died.

How to interpret a FAIL

- If **only** the rate-limiting assertion is SKIP (not FAIL) for all three endpoints, that is the expected, honest, currently-committed state – it reflects the real absence of rate limiting, not a broken challenge.
- If boba_jackett’s crash-resistance assertion FAILs occasionally, that is the tracked, real, cold-start amplification finding above – not the challenge itself being flaky. Re-run it; if it now PASSES, the endpoint had already been “warmed” by a prior probe (including this challenge’s own earlier tiers, or another test suite/health-checker hitting it recently).

- Any FAIL on merge_search or qbittorrent™'s crash-resistance or cross-endpoint-isolation assertions is unexpected and should be investigated per Â§11.4.102 (systematic debugging) before assuming it is environmental.
- SKIP for "endpoint unreachable" (reason services_not_running) means the boba stack was not running when the challenge executed " start it via ./start.sh and re-run.

Running manually

```
# Full live run against whatever boba services are currently reachable at
# 127.0.0.1:7185 / :7187 / :7189.
```

```
bash challenges/scripts/ddos_resilience_challenge.sh
```

```
# RED mode " reproduce the "no rate limiting configured" defect state
# explicitly (should PASS today; will start FAILing once a rate limiter is
# wired and later stripped).
```

```
RED_MODE=1 bash challenges/scripts/ddos_resilience_challenge.sh
```

```
# Self-validation " prove the crash-resistance detector distinguishes a
# genuinely broken (golden-bad) backend from a genuinely healthy
# (golden-good) one, using throwaway local fixtures only.
```

```
bash challenges/scripts/ddos_resilience_challenge.sh --self-validate
```

```
# --healthz " BOB-112 regression guard (added task #74): a bounded 2-thr
# x 20-connection x 5s wrk load (falls back to a curl-loop probe if wrk is
# not installed) against boba-jackett's :7189/healthz, asserting the
# client-observed timeout rate + throughput + server-side cache-refresh
# count all stay within thresholds calibrated against the real RED/GREEN
# evidence in docs/qa/BOB-112/summary.md. FAILs if the TTL cache is ever
# removed or bypassed again.
```

```
bash challenges/scripts/ddos_resilience_challenge.sh --healthz
```

Exit codes: 0 = all PASS (honest SKIPS allowed), 1 = one or more real FAIL, 2 = invocation error (curl itself is missing).

Followups filed

Originally documented here rather than inserted into docs/workable_items.db directly " this task ran in parallel with other subagents also touching shared project state, and a concurrent SQLite write from this task risked a race with theirs. Registered into the DB (Â§11.4.93/Â§11.4.202) by the orchestrating session once the parallel batch completed " see the BOB-NNN id on each item below:

1. **Bug** (filed as **BOB-112, FIXED + live-verified 2026-08-18 " task #74**): boba-jackett™'s /healthz synchronously called Jackett.GetCatalog() uncached on every hit (qBitTorrent-go/internal/jackettapi/health.go:60-63), causing up to 65% request timeout rates under a modest cold-start concurrent burst (measured evidence above). Fixed by commit 91b52db (30s TTL cache); see docs/

- qa/BOB-112/summary.md for the real wrk before/after evidence (97.1% â†’ 0.0% timeouts) and the standing --healthz regression guard this challenge now ships.
2. **Task** (filed as **BOB-111**): configure real rate limiting for boba™s three public HTTP endpoints (:7185, :7187, :7189) â€” none exists today. Candidate approaches documented above.
 3. **Task** (NOT filed as a separate BOB-NNN this round â€” remains an open scoping note here, distinct from the orchestrating session™s filed batch): extend `ddos_resilience_challenge.sh` (or add a sibling challenge) to exercise the real `POST /api/v1/search` fanout path via a sandboxed/mocked-tracker setup, without ever touching real third-party tracker sites.
 4. **Task** (filed as **BOB-114**): add a golden-bad synthetic fixture to `--self-validate` that validates the rate-limiting detector itself (currently only the crash-resistance detector is self-validated).
 5. **Task (test-type matrix, see docs/testing/test_type_matrix.md)** (filed as **BOB-109** scaling / **BOB-110** UX): scaling-class and UX-class test coverage are both fully absent from boba™s mandated test-type matrix â€” filed as separate followups there.
 6. **Task** (filed as **BOB-113, CLOSED 2026-08-18 â€” task #75/#74**): wrk was not installed on the development host (only ab was) â€” `scripts/install-dev-tools.sh` (commit `c7dfdde`) added a dev-tooling installer; it was run live during task #74 (wrk 4.2.0 [epoll] and hey confirmed ALREADY-PRESENT/installed, ab/curl-loader detected, siege an honest documented SKIP).